

Bucket Getter: A Bucket-based Processing Engine for Low-bit Block Floating Point (BFP) DNNs

Yun-Chen Lo
National Tsing Hua University
Hsinchu, Taiwan
yunchen.lo@gapp.nthu.edu.tw

Ren-Shuo Liu
National Tsing Hua University
Hsinchu, Taiwan
renshuo@ee.nthu.edu.tw

ABSTRACT

Block floating point (BFP), an efficient numerical system for deep neural networks (DNNs), achieves a good trade-off between dynamic range and hardware costs. Specifically, prior works have demonstrated that BFP format with 3~5-bit mantissa can achieve FP32-comparable accuracy for various DNN workloads. We find that the floating-point adder (FP-Acc), which contains modules for normalization, alignment, addition, and fixed-point-to-floating-point (FXP2FP) conversion, dominates the power and area overheads, hence hindering the hardware efficiency of state-of-the-art low-bit BFP processing engines (BFP-PE).

To mitigate the identified issue, we propose **Bucket Getter**, a novel architecture implemented with the following techniques for improving the energy efficiency and area efficiency: 1) we propose a bucket-based accumulation unit prior to FP-Acc, which uses multiple small accumulators (buckets) that are responsible for a small range of exponent values where intermediate results are distributed accordingly, and b) accumulate in FXP domain. This reduces the activities of power-hungry a) alignment and b) format conversion units. 2) We propose inter-bucket carry propagation, which allows each bucket to transmit overflow to an adjacent bucket and further reduces the activity of FP-Acc. 3) We propose an out-of-bound-aware, adaptive and circular bucket accumulator to significantly reduce the overhead for the bucket-based accumulator. 4) We further propose shared FP-Acc, which exploits the low activity of FP-Acc in the bucket-based architecture and shares an FP-Acc across several MAC engines to reduce the area overhead of FP-Acc.

The experimental results based on TSMC 40 nm demonstrate that our proposed Bucket Getter architecture reduces the computational energy by up to 57% and improves the area efficiency by up to 1.4 $\times$, compared to state-of-the-art BFP engines across seven representative DNN models. Furthermore, our proposed approach helps state-of-the-art floating-point engines reduce up to 32% of the PE area and 81% of the PE power.

CCS CONCEPTS

• **Computer systems organization** → **Neural networks**; • **Hardware** → **Arithmetic and datapath circuits**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0329-4/23/10...\$15.00
<https://doi.org/10.1145/3613424.3614249>

KEYWORDS

Floating-point architecture, Bucket-based accumulation, Deep learning

ACM Reference Format:

Yun-Chen Lo and Ren-Shuo Liu. 2023. Bucket Getter: A Bucket-based Processing Engine for Low-bit Block Floating Point (BFP) DNNs. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3613424.3614249>

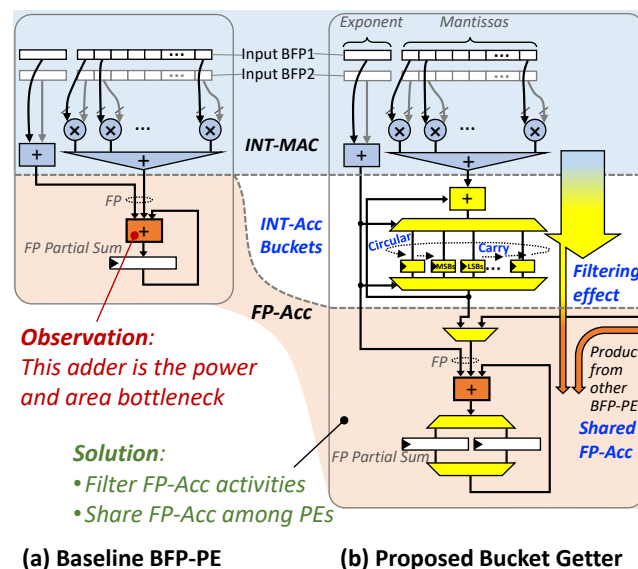


Figure 1: Big picture of the proposed architecture

1 INTRODUCTION

Deep neural networks (DNNs) have been continuously enabling many new applications [30, 32, 34, 48], which has lead to the clear demand for deploying an enormous number of DNN accelerators into cloud servers [2, 19] and edge devices [16, 18, 24, 27, 28]. To push the limit of energy efficiency and cost efficiency for DNN accelerators, researchers have explored custom number systems and hardware [7, 20, 23, 46]. Among them, custom, low-bit floating point (FP) formats, including block floating point (BFP), are promising candidates. For example, BrainFloat (also known as bfloat or BF16) [20] and TensorFloat [33] are low-bit FP formats tailored for DNN accelerators by Google and NVIDIA, respectively. BrainFloat comprises one sign bit, seven mantissa bits, and eight exponent

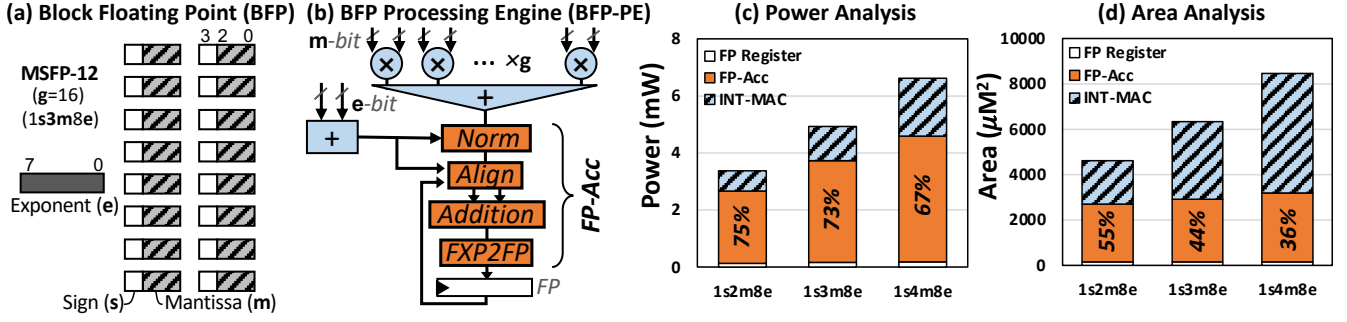


Figure 2: (a) Block floating point format (MSFP-12), (b) State-of-the-art BFP processing engine (BFP-PE) [7, 37], (c) Power analysis of SOTA BFP-PEs, and (d) Area analysis of SOTA BFP-PEs

bits (1s7m8e for short), and TensorFloat comprises 1s10m8e bits. Since DNNs involve a massive number of inner products among vectors, the block floating point (BFP) [7, 8, 21, 37, 43] format encodes each vector of, e.g., 16 FP numbers into 16 sign-magnitude integers with one shared, factored-out exponent. By doing so, an inner product between two BFP vectors involves multiplications of sign-magnitude integers instead of that for FP numbers. Specifically, Microsoft’s BFP, or MSFP, as coined by Microsoft, is employed in Microsoft’s Brainwave inference accelerators and demonstrates promising power and cost efficiency [7, 10]. Among many configurations, one representative MSFP configuration is MSFP-12 (i.e., 1s3m8e, where $1 + 3 + 8 = 12$), which packs 16 4-bit sign-magnitude integers (i.e., 1s3m) with one shared 8-bit exponent (i.e., 8e) to represent a 16-tuple FP vector.

In response, this work focuses on designing accelerators for this emerging DNN-friendly format, BFP (also MSFP). Specifically, we provide insights into the power and hardware bottlenecks of BFP-based processing engines (BFP-PE) that perform inner products, and we correspondingly improve the microarchitecture of the PEs. As shown in Figure 1, we identify a critical adder that plays a bottleneck role in terms of power and area in a BFP-PE. Therefore, we propose to lower the power consumption of the adder by introducing a layer of buckets, which filter out the majority of activities reaching the adder, similar to a cache in a memory hierarchy. In addition, since the number of adder activities is reduced, we propose sharing one adder among multiple PEs to reduce the overall hardware cost (i.e., area).

Specifically, a baseline BFP-PE (Figure 1(a)) is composed of two stages: INT-MAC (integer multiply and accumulate) and FP-ACC (FP accumulate). The INT-MAC stage involves low-bit sign-magnitude multipliers, an adder tree, and an integer adder for the exponents. We select MSFP-12, as an example. The INT-MAC stage comprises 16 4-bit multipliers and an adder tree with 16 7-bit inputs to reduce the 16 products to a partial-sum term. The FP-ACC stage first scales the partial-sum term according to the sum of the two exponents. Since DNN involves MAC with thousands of products (e.g., a $3 \times 3 \times 512$ DNN kernel leads to 4,608 products), the partial-sum term has to be accumulated with other previously computed partial-sum terms in an FP accumulation register.

One may think that the baseline BFP-PEs are sufficiently optimized since BFP format narrows the mantissa fields from 23 bits in

conventional FP format to approximately 3 bits (e.g., 1s3m) and only needs to employ one exponent per, e.g., 16 FP numbers. Although this impression is not totally incorrect, we observed a less intuitive finding: the FP-Acc stage, which still involves FP operations, remains a power and area bottleneck.

Figure 2 quantitatively shows the bottleneck. We use Design Compiler to synthesize BFP-PEs and use PrimeTimePX to break down their power consumption on real workloads with TSMC 40 nm technology. We analyze three BFP formats, namely, 1s2m8e (MSFP-11), 1s3m8e (MSFP-12), and 1s4m8e (MSFP-13). Figure 2(a) only illustrates a representative case, 1s3m8e (MSFP-12), for brevity. Figure 2(b) shows the detailed BFP-PE structure and emphasizes the FP-Acc stage. Unlike in Figure 1, in which FP-Acc is illustrated as an adder for brevity, the fact is that the FP-Acc involves multiple units. In addition to the adder, FP-Acc further requires normalization, alignment, and fixed-point-to-floating-point conversion (FXP2FP) units. It is because of these many units that FP-Acc contributes to significant power and area consumption in a BFP-PE.

Figure 2(c) shows the power breakdown of a BFP-PE. The FP-Acc stage contributes to 75%, 73%, and 67% of the PE power for the 1s2m8e, 1s3m8e, and 1s4m8e configurations, respectively. Clearly, FP-Acc dominates the power consumption of a BFP-PE. Figure 2(d) shows the area breakdown of a BFP-PE, and FP-Acc contributes to 36%, 44%, and 55% of the PE area for the 1s2m8e, 1s3m8e, and 1s4m8e configurations, respectively. In other words, one adder in the FP-Acc stage can lead to a larger area than the adder tree and 16 multipliers in the INT-MAC stage.

We point out the rationales for why FP-Acc contributes to significant power and area as follows.

- (1) The normalization unit must find the leading one of the adder tree output for the subsequent alignment. A leading-one detector utilizes a tree structure and thus incurs large power and area costs [31].
- (2) The addition unit is in 23-bit precision (i.e., a single-precision FP) instead of low-bit precision.
- (3) The FXP2FP unit also requires a leading-one detector. Furthermore, FXP2FP requires adders and logic to format the final FP number.

In response, we propose **Bucket Getter** architecture to enhance the energy efficiency and area efficiency of BFP-PEs. To mitigate the

large power overhead of FP-Acc in BFP-PEs, we propose **bucket-based accumulator architecture** to reduce the activity of FP-Acc. Specifically, instead of performing accumulation using just one register in the FP domain, the bucket-based accumulator utilizes a group of small accumulators (buckets), and each of them is only responsible for accumulating partial sums with a small range of exponent values. Since each bucket is only responsible for accumulating partial sums within a limited dynamic range, the circuit complexity is significantly reduced. The architectural benefits of bucket-based accumulators are twofold: 1) Accumulation using a small bucket accumulator eliminates the need for normalization and FXP2FP since a partial sum can be accumulated in the FXP domain. Therefore, the power overheads of these modules are reduced. 2) Since each bucket is only responsible for a small range of exponent values, the shifter alignment unit can be greatly simplified, hence reducing its power consumption. We further propose two mechanisms, namely, *inter-bucket carry propagation* and *adaptive circular bucket architecture*, to optimize the area and power overhead. To further reduce the size of FP-Acc, we propose *shared FP-Acc*. By exploiting the low activity of FP-Acc in the bucket-based architecture, we can share FP-Acc among several PEs to reduce the area overhead of FP-Acc.

In summary, the key contributions of this work are as follows:

- We identify that the power and area bottlenecks of state-of-the-art BFP-PEs are floating-point accumulation (FP-Acc), which contributes up to 75% of the power and 55% of the area costs of BFP-PEs.
- We propose a bucket-based BFP-PE architecture, which reduces the power consumption by replacing most of the FP-Acc activities with INT-Acc activities.
- We propose an inter-bucket carry propagation scheme that allows each bucket to transfer overflow to an adjacent bucket to further reduce the activity of FP-Acc.
- We propose an out-of-bound-aware adaptive circular bucket accumulator to reduce the required number of buckets.
- We propose a shared FP-Acc module, which shares an FP-Acc across multiple PEs to reduce the area overhead of FP-Acc.
- Experiments based on seven DNN models demonstrate that Bucket Getter outperforms state-of-the-art BFP-PEs by up to 57% in terms of the computational energy reduction and improves iso-area speedup by up to 1.4×

The rest of this paper is organized as follows: Section 2 presents the benefits of the bucket-based architecture; Section 3 illustrates the bucket-based accumulation strategy; Section 4 presents the inter-bucket carry propagation scheme; Section 5 introduces the out-of-bound-aware adaptive and circular bucket accumulator; Section 6 presents the shared FP-Acc technique; Section 7 presents the Bucket Getter accelerator; Section 8 describes how to support our techniques for FP-PEs; Section 9 presents the potential impact of the design; Section 10 presents the experimental results; Section 11 discusses the related works; Finally, Section 12 concludes this work.

2 BENEFITS OF THE BUCKET-BASED ACCUMULATION STRATEGY

We first compare the bucket-based accumulation strategy with the conventional FP accumulation strategy to showcase the benefits of

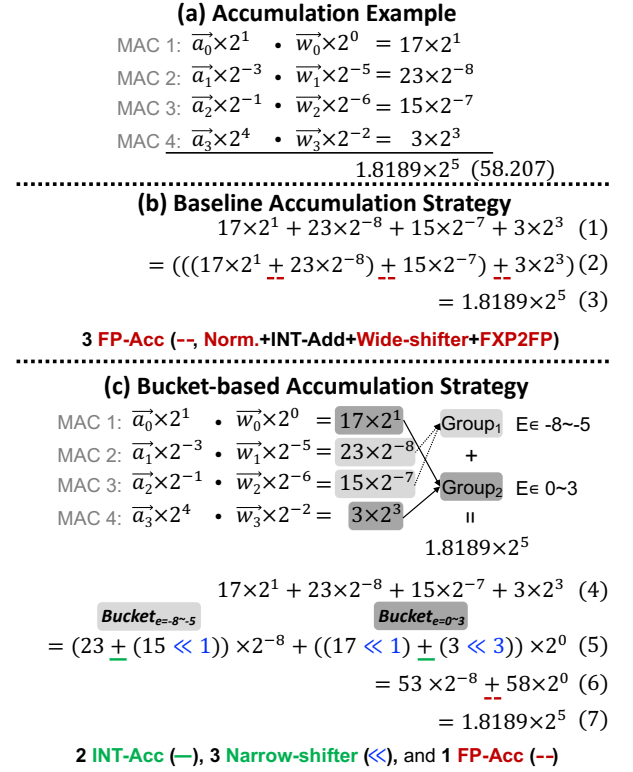


Figure 3: (a) An example of accumulating 4 BFP partial sums. (b) The baseline accumulation method [7, 37] directly performs FP-Acc, which is energy-consuming. (c) We classify the partial sums into groups based on the exponent values. The bucket-based accumulation approach yields partial sums with similar exponent values in the same buckets, hence reducing FP-Acc activity to one.

our proposal. Figure 3(a) presents an example of accumulating four partial sum terms of BFP. We assume that each bucket is responsible for a range of four exponent values. Therefore, we assume the groups to be [-8 to -5], [-4 to -1], and [0 to 3]. Figure 3(b) shows the baseline strategy adopted by state-of-the-art BFP-PEs [7, 37], and in this approach, FP-Acc is performed for each cycle. To accumulate four partial sum terms, baseline accumulation requires 3 FP-Accs. The key observation of this work is that we can separate the four partial sums in Figure 3(a) into two groups, where group 1 and group 2 are responsible for exponent values of 0 ~ 3 and -8 ~ -5, respectively. As shown in Figure 3(c), the dynamic range of each group is small (4 bits); hence, accumulating these terms requires simple operations. Specifically, we can perform accumulation in the FXP domain within each group of partial sums, hence eliminating the power-hungry FP-Acc. Equations (4)~(7) show that we can generate the same numerical results while reducing FP-Acc activity by $\frac{2}{3}$ (67%); additionally, only one FP-Acc is needed to reconstruct the FP partial sum back to the FP-domain. In the following section, we present the corresponding bucket-based accumulator.

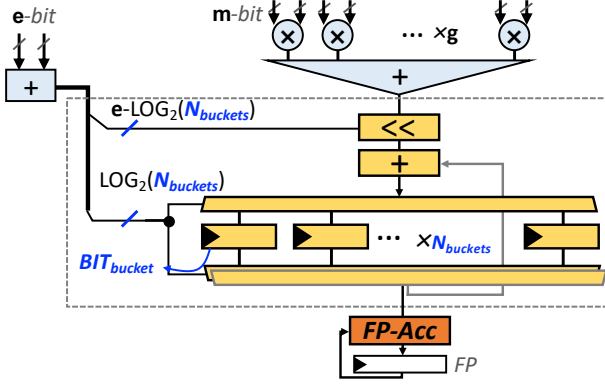


Figure 4: BFP-PE using a bucket-based accumulator (highlighted with yellow)

3 BUCKET-BASED ACCUMULATOR

Figure 4 shows the bucket-based accumulator and the corresponding PE. We highlight the key modules, namely the narrow shifter with several bucket accumulation registers, in yellow (we use $N_{buckets} \times BIT_{bucket}$ to denote a configuration with $N_{buckets}$ buckets, and each bucket has a bitwidth of BIT_{bucket} in the following paragraph). These buckets are inserted between the MAC and FP-Acc to independently accumulate the partial sums that fall into different ranges of exponent values. The corresponding bucket is automatically selected using the MSBs of the exponent sum. Accumulation using these buckets significantly reduces the activity of power-hungry normalization and FP-Acc, hence reducing the overall power. Specifically, Figure 5 compares the power (analyzed using PTPX) of computing several real BFP vectors of ResNet-18 using the baseline BFP-PE [7] and the proposed bucket-based BFP-PE. Notably, the bucket-based BFP-PE reduces power consumption by 65% compared to the baseline BFP-PE.

To better understand our proposed approach, we point out that the relationship between the first-stage bucket accumulator and second-stage FP accumulator is analogous to a level 1 (L1) cache and a level 2 (L2) cache in classic computer systems in the following ways:

- (1) The bucket accumulator consumes less energy than the FP accumulator but may need support from the FP accumulator when it encounters overflow. Similarly, the L1 cache consumes less energy per access but requires support from the L2 cache when the working set size exceeds its physical size.
- (2) Most of the accumulation process only involves the bucket accumulator, and only a small proportion of accumulation requires the FP accumulator. Similarly, most of the memory accesses utilize the L1 cache, and only a small number of memory accesses require the L2 cache.

Furthermore, two important architectural parameters are relevant to bucket-based PEs: $N_{buckets}$ and BIT_{bucket} . We discuss the impact of adjusting these architectural parameters below:

Number of buckets ($N_{buckets}$): The number of buckets influences the number of exponent values associated with each bucket ($N_{exponent}$). For example, architecture with four buckets requires

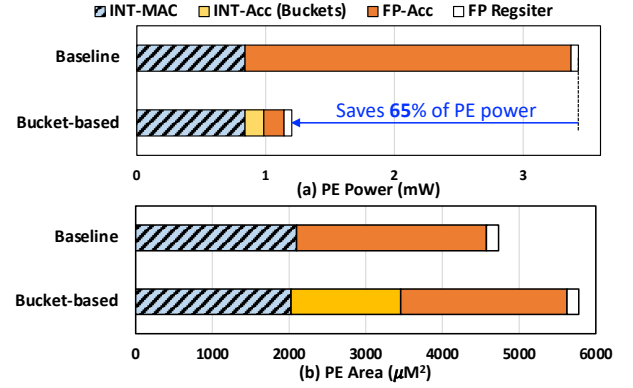


Figure 5: (a) Bucket-based architecture ($16 \times 12b$ buckets), which reduces power consumption by up to 65% (Prime-TimePX @ 500 MHz) compared to that of the baseline computing blocks of ResNet-18. (b) Bucket-based accumulator, which reduces the area by 20% via the use of buckets. Our following proposed approach further considers the out-of-bounds concept to mitigate the area overhead.

each bucket to be responsible for $256/4 = 64$ exponent values. On the other hand, in the proposed architecture, employing 32 buckets makes each bucket handle only $256/32 = 8$ exponent values.

Since increasing $N_{buckets}$ reduces $N_{exponent}$, it reduces the bitwidth required by bucket registers. In other words, it reduces the overflow possibility of buckets given the same bucket bitwidth BIT_{bucket} . However, more buckets also lead to larger area and power overhead compared to that with fewer buckets.

Bitwidth of each bucket (BIT_{bucket}): The bitwidth of the buckets influences the bucket power, bucket area, and activity of the FP-Acc unit. Specifically, increasing the bucket bitwidth reduces the activity of the power-hungry FP-Acc unit at the cost of incurring larger bucket power and area overhead.

To determine an optimized bucket configuration in terms of area and power, we perform detailed design space exploration using a real workload. Please refer to Figure 12 in Section 10.2 for more details.

4 INTER-BUCKET CARRY PROPAGATION

One shortcoming of bucket-based accumulators is that they still require significant FP-Acc activity in cases with a low hardware budget. (i.e., limited bucket bitwidth BIT_{bucket} and a small number of buckets N_{bucket}). To address this issue, we propose inter-bucket carry propagation to transfer overflow between buckets and hence reduce FP-Acc activity.

The naive bucket architecture and its overflow handling procedure only consider each bucket to independently accumulate partial sums in a small range of exponent values. Thus, an overflowed bucket needs to trigger FP-Acc immediately and flush its partial sum to maintain numerical correctness. As shown in Figure 6(a), when overflow occurs for the yellow bucket accumulator, other buckets in gray cannot assist the yellow bucket, hence triggering significant FP-Acc activity.

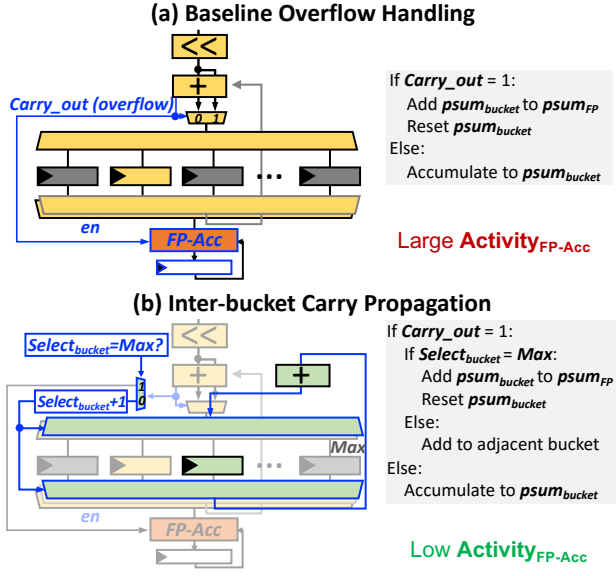


Figure 6: (a) Bucket-based accumulator and baseline overflow handling procedure. (b) The proposed inter-bucket carry propagation scheme, which adds carry-out to the adjacent bucket accumulation register and further reduces the activity of FP-Acc. This technique reduces FP-Acc activity by up to 86%, as shown in Figure 12.

To reduce FP-Acc activity, we propose inter-bucket carry propagation, as shown in Figure 6(b). We enable the bucket to propagate its carry-out (i.e., overflow) to the bucket to its right when facing overflow conditions. To support this approach, we add an additional set of multiplexer and demultiplexer circuitry and the corresponding right-hand bucket selection logic. This hardware support helps to add the overflow term to the designated bucket, which is shown as the green bucket in Figure 6(b). Please refer to the pseudocode for the details of the inter-bucket carry propagation control flow.

5 ADAPTIVE AND CIRCULAR BUCKETS

One unique and exploitable feature of FP-Acc is to skip out-of-bound (OB) partial sums [1, 4, 29]. As shown in Figure 7(a), adding a significantly small partial sum to the original FP partial sum does not affect the result in the IEEE-754 standard due to the limited output mantissa field, e.g., $1.5 \times 2^2 + 1.1 \times 2^{-24} \rightarrow 1.5 \times 2^2$.

Figure 7(b) shows the benefits of exploiting out-of-bound FP-Acc in a static bucket accumulator. Specifically, after completing bucket accumulation, the OB-aware static bucket architecture first identifies the current largest-value bucket to determine in-bound and out-of-bound buckets. Then, it skips adding out-of-bound buckets to the FP accumulation register to reduce FP-Acc activity. However, since the static bucket does not know the current largest-value bucket beforehand and each bucket is responsible for a fixed range of exponent values, it still requires maintaining all of the buckets and hence yields significant area and power overhead. For example, we need to allocate 64 buckets in Figure 7(b) if each bucket is responsible for four exponent values.

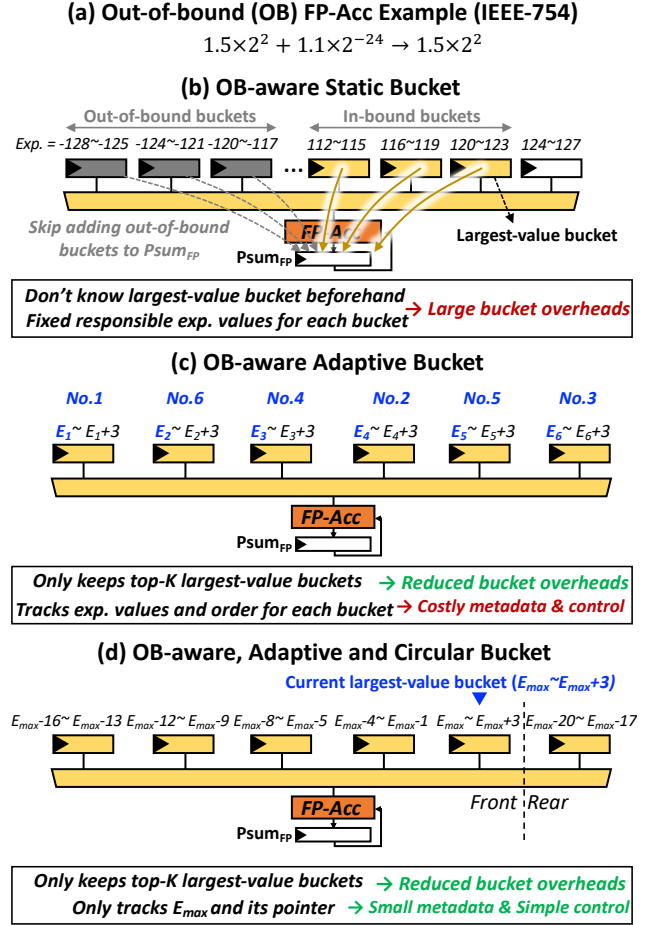


Figure 7: (a) Out-of-bound (OB) FP-Acc example. (b) OB-aware static bucket architecture. (c) OB-aware adaptive bucket. (d) OB-aware, adaptive and circular bucket architectures. We denote the metadata using blue text. In general, considering out-of-bounds cases consistently reduces the PE area by 21.19 ~ 71.72%.

In response, Figure 7(c) shows the proposed OB-aware adaptive bucket accumulator. The high-level concept is to track the responsible exponent values for each bucket and keep only the top-K in-bound buckets. During the execution process, we check whether the upcoming partial sum is larger than the current largest-value bucket. We then check the bucket order and replace the partial sum to store the largest one. For the same example in which each bucket is responsible for four exponent values, we only need to allocate six buckets to cover the exponent values that contribute to the final 24-b mantissa of FP partial sum. In this example, our design reduces the area cost by up to 70.6%.

Although we significantly reduce the bucket area, this design still requires costly metadata and complex bucket control logic. To mitigate this issue, we propose organizing all in-bound buckets in a circular fashion. Figure 7(d) shows the benefits of the circular bucket approach, where we only need to memorize the position of

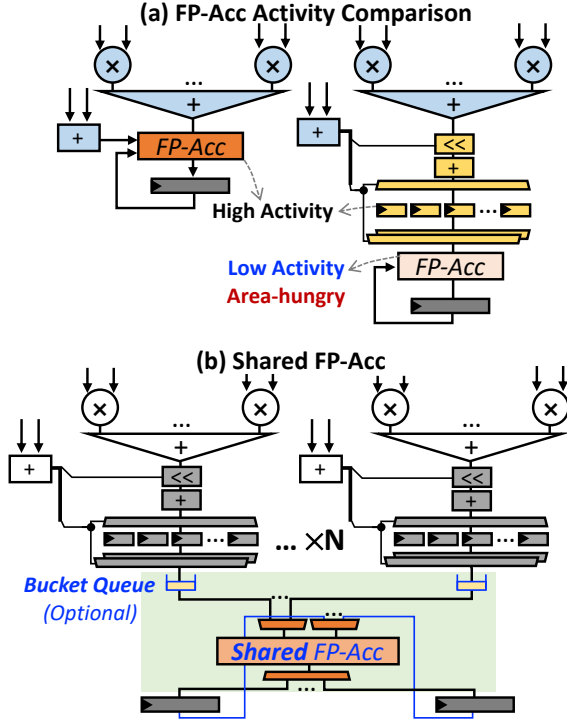


Figure 8: (a) FP-Acc activity comparison for the baseline and bucket-based PEs. The bucket-based architecture reduces FP-Acc activity, which confirms the effectiveness of the proposed approach. (b) The proposed shared FP-Acc. Sharing one FP-Acc across two PEs reduces the area by by 29.8%.

the front bucket and the corresponding exponent values to infer the exponent ranges for other buckets. When all of the buckets are empty and an upcoming partial sum needs to be accumulated, we randomly select a bucket for storage and configure this bucket at the front of the circular queue. When there is an upcoming partial sum with an exponent value larger than the existing largest exponent, we let it occupy the space of the smallest bucket. We delete the smallest bucket's partial sum if this partial sum is out-of-bound.

6 SHARED FLOATING-POINT ACCUMULATION (SHARED FP-ACC)

One unique feature of the bucket-based architecture is the low activity of the FP-Acc unit after including a bucket-based accumulator. Figure 8(a) shows the activity of each accumulator, and the relative activity levels are shown. Our empirical results show that FP-Acc only activates in 4 ~ 10% of the total cycle among various workloads. The identified characteristic helps to mitigate the area overheads of the FP-Acc unit, which occupies up to 53% of the total PE area.

Specifically, we propose exploiting the low activity of the FP-Acc module and sharing one FP-Acc module across different PEs. Figure 8(b) shows two designated hardware modifications to enable

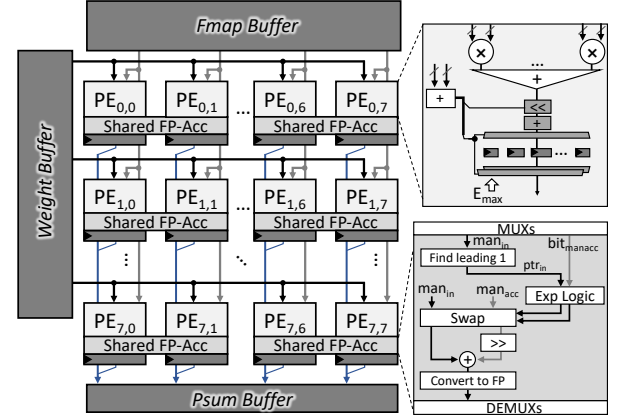


Figure 9: Bucket Getter accelerator

the shared FP-Acc: 1) We add two multiplexers and one demultiplexer to configure the input and output of the FP-Acc unit, which logically becomes a part of any PEs. In other words, one physical FP-Acc can be assigned to accumulate the partial sum for the buckets of multiple PEs. 2) We include an option to add bucket queues between the bucket accumulator and the shared FP-Acc to avoid potential performance degradation if extreme workloads occur. Our empirical results (Figure 15) show that sharing one FP-Acc among two PEs provides a significant area improvement for a representative workload, ResNet-18. Therefore, our final design is based on $N_{shared} = 2$.

7 BUCKET GETTER ACCELERATOR

Now, we integrate all of the proposed techniques to complete a Bucket Getter tile, as shown in Figure 9. First, we organize a tile as a 2D PE array to allow spatial data reuse across different PEs, where feature map blocks are shared vertically, and weight blocks are shared horizontally. In this way, the power of SRAM accesses is mitigated. This array configuration is also adopted in several state-of-the-art papers [4, 41].

For each PE, we allocate a 16-unit-wide MAC and an adaptive and circular bucket-based accumulator with inter-bucket carry propagation capability. Regarding the FP accumulator, we allow two adjacent PEs to share an FP-Acc to reduce the area overhead. This configuration reduces the area and maintains a certain design margin that prevents performance degradation. Finally, the partial sum is accumulated using the Psum buffer.

7.1 Execution Example

Figure 10 shows the required steps to compute eight feature map (fmap) blocks with eight weight blocks using two bucket-based PEs in Bucket Getter, where the channel number is 64 and the block size is 16. We denote different mantissa blocks of the feature map as $\alpha \sim \theta$. Similarly, we denote different mantissa blocks of weights as $a \sim l$. The block-wise exponent terms are marked separately. For each PE, we allocate two buckets, where each bucket is responsible for four exponent values. In addition, we share one FP-Acc across two PEs and allocate a bucket queue with a depth equal to one for

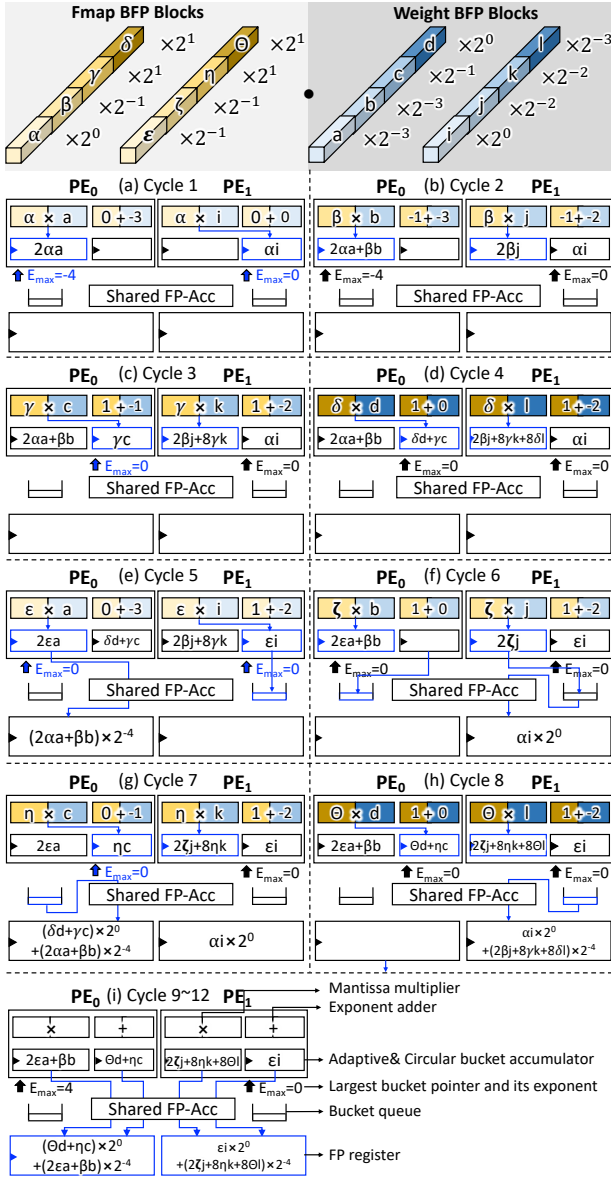


Figure 10: Cycle-by-cycle execution example of computing eight feature map blocks and eight weight blocks with two bucket-based PEs. We highlight the activated modules and data path using blue.

each PE. An FP-Acc task is assumed to be completed using one MAC cycle in this example.

Here, we recap that there are two main phases of computation involved: bucket-based accumulation and MAC reconstruction. The former phase accumulates the partial sum using bucket accumulators and significantly reduces the power. The latter phase includes dumping accumulated partial sums from the bucket accumulators to the FP accumulator, which is only triggered when bucket overflow is encountered for the largest-value bucket or when a group

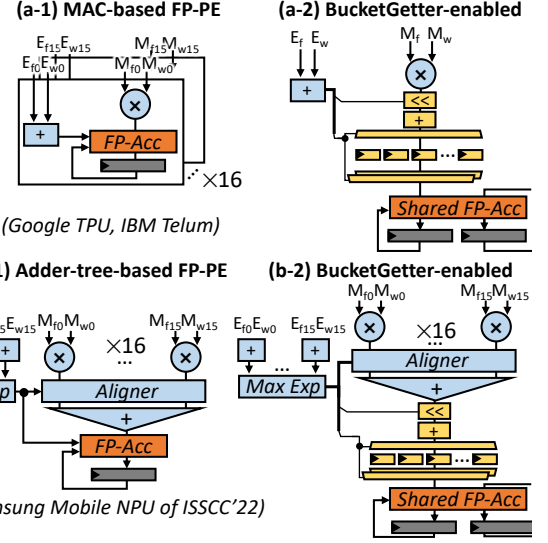


Figure 11: (a-1) MAC-based FP-PE and (a-2) its BucketGetter-enabled design. (b-1) Adder-tree-based FP-PE and (b-2) its BucketGetter-enabled design.

of accumulations is completed. Please note that these two phases can be performed in parallel without stalling each other.

Initially, the PE accumulators are empty. In cycle 1, two PEs multiply fmap block α by weight blocks a and i to produce corresponding partial sums, αa and αi , respectively. The partial sum αa of PE_0 is scaled by 2 (left shift 1 bit) because its exponent sum is -3 and has to be stored in the bucket that is responsible for exponents of $-4 \sim -1$. E_{max} is set to -4, and we set the left bucket as the current largest-value bucket. For PE_1 , the partial sum αi does not need to be scaled because the exponent sum equals the lower bound of the bucket. Then, we randomly select the right-hand bucket as the current largest-value bucket and set E_{max} to 0.

In cycle 2, we multiply fmap block β by weight blocks b and j to produce partial sums, βb and βj , respectively. PE_0 accumulates the partial sum βb using its left-hand bucket. PE_1 scales βb by 2 (left shift 1 bit) and accumulates the partial sum to the left-hand bucket, which is responsible for exponent values of $-4 \sim -1$.

In cycle 3, the same procedure is used. The only difference is that the upcoming partial sum γc of PE_0 yields a larger exponent value than that for the current bucket; i.e., 0 is greater than -1. Thus, we move the largest bucket pointer to the right-hand bucket and adjust the corresponding exponent value E_{max} . Finally, the first four blocks complete their partial sum computation and accumulation processes in cycle 4.

In cycle 5, the upper part of two PEs multiplies fmap block ϵ belonging to different pixel positions, again by weight blocks a and i , to produce corresponding partial sums, ϵa and ϵi , respectively. These partial sums are required to be stored in empty buckets. Thus, the old buckets have to be flushed to the FP accumulator. The shared FP-Acc is configured to flush the partial sum of the left-hand bucket in PE_0 , which frees up space for the upcoming partial sum. Since

the shared FP-Acc is busy, PE_1 flushes the partial sum to the bucket queue and frees up its register for the upcoming partial sum.

Similarly, in cycles 6~8, the new partial sums are computed and the previous partial sums are simultaneously flushed to the FP accumulator. After all pixel computations are complete, we clean up the registers and reconstruct the FP partial sum in cycles 9~12. Please note that the cycle overhead for cleaning up the buckets is insignificant because modern DNN workloads involve many input pixels. We can even hide the corresponding cycle overhead if we compute the next layer in parallel.

8 EXTENSION TO FLOATING-POINT PES

We emphasize that the proposed bucket-based accumulator and optimizations can be easily integrated into FP processing engines (FP-PE), which further broadens this work's impact. We showcase how to integrate our proposed approach into state-of-the-art FP-PEs [1], which do not share an exponent across a block of mantissa words.

Figure 11(a) shows the MAC-based FP-PE and its corresponding BucketGetter-enabled design. The MAC-based FP-PE multiplies two operands and accumulates the partial sum using an FP-Acc, an approach adopted by IBM Telum AI processor [2, 47] and Google TPU [19]. Furthermore, we show that it is straightforward to insert a bucket accumulator between the multiplier and FP-Acc.

Figure 11(b) shows an adder-tree-based FP-PE and the corresponding bucket-based architecture. The adder-tree-based FP-PE first multiplies 16 mantissas and aligns their partial products with the per-vector maximum exponent value. It then utilizes an FP-Acc to generate an updated version of the FP partial sum. This architecture enables high performance and is utilized in Samsung's flagship mobile SoC [36]. Similarly, we show that it is easy to insert a bucket-based accumulator to reduce power consumption.

9 POTENTIAL IMPACT

The proposed Bucket Getter schemes can generally improve the area and energy efficiency for BFP or FP PEs without approximation, supporting integration with numerous DNN accelerators. Since FP and BFP can be utilized during both the training phase [8, 20] and the inference phase [7, 46], our schemes have the potential to improve the efficiency for both phases of DNNs. Specifically, we perform PrimeTimePX power analysis using an actual pattern when training a ResNet and find that the bucket-based accumulator reduces the PE power by 45.9%, which shows promising potential for training.

10 EVALUATION

10.1 Experimental Setup

DNN workloads: We perform experiments on seven representative DNNs: ResNet-18 (Res-18), ResNet-50 (Res-50), MobileNet-v1 (Mb-v1), MobileNet-v2 (Mb-v2), ShuffleNet-v2 (Sh-v2), Vision Transformer (ViT), and GPT-2. These workloads cover traditional classifiers, lightweight classifiers, transformer-based classifiers, and transformer-based text generation. We list the applications and datasets of these models in Table 1. Among them, many models (e.g., ResNet, MobileNet, and GPT) are also used by the industry-standard MLPerf benchmark [39].

Table 1: Studied models

Model	Application	Dataset
ResNet-18 (Res-18)	Image Classification	ImageNet
ResNet-50 (Res-50)	Image Classification	ImageNet
MobileNet-v1 (Mb-v1)	Image Classification	ImageNet
MobileNet-v2 (Mb-v2)	Image Classification	ImageNet
ShuffleNet-v2 (Sh-v2)	Image Classification	ImageNet
Vision Transformer (ViT)	Image Classification	ImageNet
GPT-2	Natural Language Inference	WebText

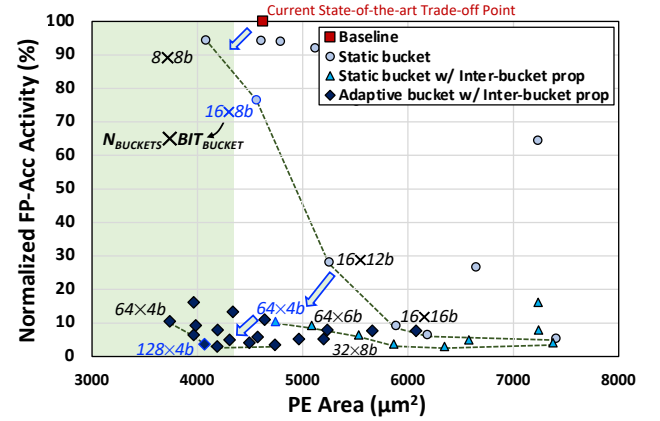


Figure 12: Design space exploration to find the optimized configuration for the optimal trade-off between the PE area and normalized FP-Acc activity

BFP configurations: We choose state-of-the-art MSFP as our baseline method. If not specified, we utilize MSFP-11 (1s2m8e) in our evaluation. We adopt pretrained FP32 checkpoints and directly apply post-training BFP quantization to obtain low-bit models. We follow the suggestions of Microsoft [7] and set the block size to 16. Furthermore, we perform additional experiments on MSFP-11 (1s2m8e), MSFP-12 (1s3m8e), MSFP-13 (1s4m8e), and MSFP-14 (1s5m8e). Note that both the hardware and software implementations were based on those in the original MSFP [7, 10] paper.

Hardware modeling: We implement all architectures using Verilog and synthesize them using TSMC 40-nm technology with the Synopsys Design Compiler (topographical mode). We synthesize all designs with a 500 MHz clock frequency and verify them using real test patterns. We utilize PrimeTimePX with real ResNet-18 test patterns to analyze the module-wise power. We then implement a simulator extending the PyTorch framework to analyze the module-wise activity. Regarding the simulator, we extend PyTorch's convolution and matrix multiplication procedure and employ variables to model the bucket accumulators. We employ some additional statistical variables to track the module-wise activities. Finally, the module-wise power analysis is combined with the simulator activity information to obtain energy estimates.

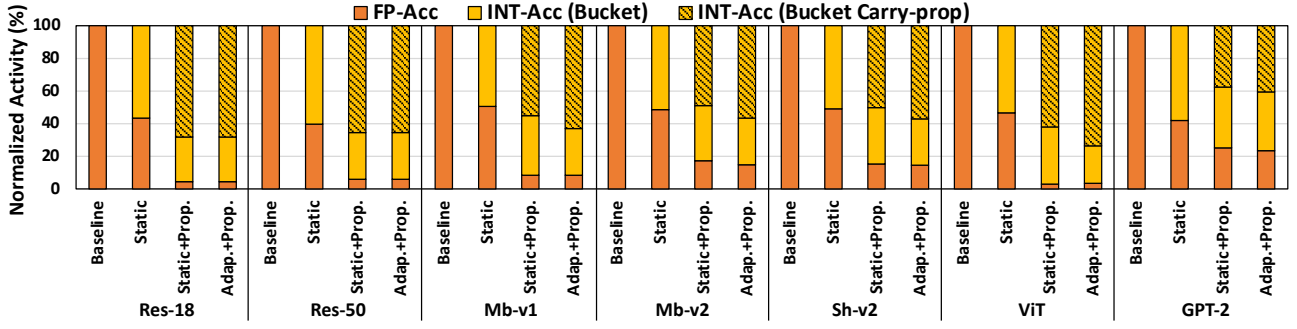


Figure 13: Accumulator activity breakdown for seven representative models

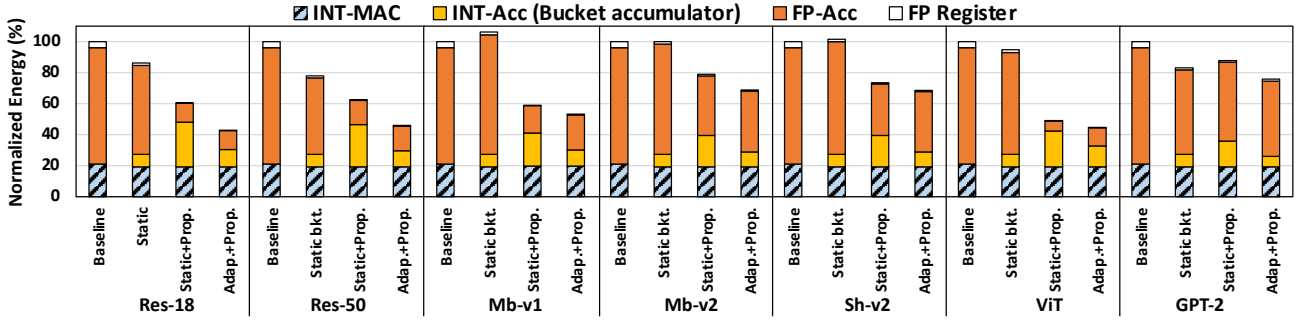


Figure 14: Normalized energy comparison of different schemes based on seven representative models

10.2 Determining the Optimized Bucket Setup

Figure 12 shows the design space exploration of different techniques and configurations. Specifically, we sweep the bitwidth of the bucket BIT_{bucket} and the number of buckets $N_{buckets}$ to find the trade-off point between PE area and FP-Acc activity based on inference with ResNet-18. We denote the baseline as a red rectangle and label the bucket-based architecture using blue dots. Specifically, light blue represents the static bucket architecture, and dark blue indicates adaptive and circular buckets. For each bucket-based architecture, we mark its configuration as $N_{bucket} \times BIT_{bucket}b$. We highlight the Pareto frontiers with dotted lines. The solution in the lower-left corner is the most ideal one, as it achieves the fewest number of FP-Acc actions with the smallest PE area.

The first takeaway is that the Pareto frontiers consistently move toward the ideal corner for different optimizations, i.e., static bucket-based architecture, inter-bucket carry propagation, and adaptive-circular bucket architecture. If we compare FP-Acc activity under iso-area conditions, we find that static bucket architecture ($16 \times 8b$) reduces the FP-Acc activities by 25%. Then, inter-bucket carry propagation additionally reduces the number of FP-Acc activities by 86%. Finally, the adaptive and circular bucket architecture reduces the number of FP-Acc activities by 66% compared to that in the static bucket architecture case with inter-bucket carry propagation.

The second takeaway is that different optimizations yield significantly different trade-off profiles. For example, the static bucket architecture produces an elbow point with a $16 \times 12b$ configuration, and inter-bucket carry propagation produces a sweep point configuration with $32 \times 8b$. Therefore, performing a simulation

and finding a suitable bucket configuration is important. In this work, for fair comparisons, we choose a configuration corresponding to the elbow trade-off point if the resulting PE area does not exceed the baseline. Otherwise, we select a configuration that is closest to the elbow trade-off point and has comparable area to the baseline. *Specifically, we set the bucket configurations for the static bucket-based architecture, inter-bucket carry propagation architecture, and adaptive-circular bucket architecture to be $16 \times 8b$, $64 \times 4b$, and $128 \times 4b$, respectively.*

10.3 Accumulation Activity Breakdown

Figure 13 shows the normalized activity of accumulators, namely, FP-Acc, INT-Acc, and INT-Acc (inter-bucket carry propagation). We summarize the accumulator activity for seven representative models. In brief, the static bucket-based architecture transforms more than 50% of the FP-Acc activities to INT-Acc. Inter-bucket carry propagation further reduces the percentage of FP-Acc activity to less than 30% of that in the initial case. Finally, the adaptive bucket architecture achieves a similar level of FP-Acc activity as that in the static bucket case, where the former is associated with lower bucket area overhead.

Let us further compare accumulator activity across different models. First, we show that the activity profiles are similar among ResNet-based models. One potential rationale for this observation is that these networks obtain similar MAC characteristics since they are in the category of traditional DNNs and involve similar network architecture. Traditional DNNs contain high redundancy and do not fully utilize the dynamic range of MAC. Second, there is

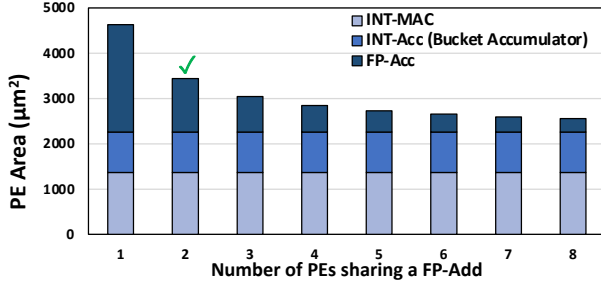


Figure 15: PE area with different numbers of PEs sharing an FP-Acc (shared FP-Acc).

a noticeable difference between traditional and lightweight models in terms of accumulator activity. Specifically, lightweight models produce a larger MAC dynamic range than conventional models and hence more FP-Acc activity. Furthermore, ViT shows a similar profile as ResNets, which is because of the high model redundancy like conventional DNNs. Finally, GPT-2 produces an even larger MAC dynamic range, hence increasing the activity of FP-Acc. This is potentially because of the large weight dynamic range [46].

10.4 Computation Energy Comparison

Figure 14 presents the normalized energy breakdown of 7 representative models based on three schemes. In general, the static bucket-based architecture transforms some FP-Acc to INT-Acc and achieves lower energy use than that in the baseline. Specifically, it reduces $\sim 20\%$ of the energy required for ResNet-18 and GPT-2, respectively. However, the static bucket architecture does not show energy reductions for MobileNet-v2, ShuffleNet-v2, and ViT for the following reasons: 1) We limit our static bucket-based architecture to utilize baseline-comparable PE area, which inevitably exclude the most energy-efficient bucket and bitwidth configurations, and 2) The large MAC dynamic range of these DNNs triggers too many FP-Acc activities due to bucket overflow. Inter-bucket carry propagation addresses the issue and further reduces the activity of FP-Acc, with comparable areas. Specifically, this approach consistently reduces the energy among all of the models by 5 $\sim$ 40%. Finally, the adaptive circular bucket architecture achieves the lowest power among all schemes because the power overhead for the bucket accumulator is further reduced. Specifically, the energy is decreased by 23 $\sim$ 57% across all models. We emphasize that energy reduction is achieved while maintaining the same level of numerical precision.

10.5 Area Reduction with Shared FP-Acc

Figure 15 analyzes the area benefit of the proposed shared FP-Acc scheme, where multiple PEs share one FP-Acc. We sweep the number of PEs sharing one FP-Acc and correspondingly present the PE area. We find that sharing one FP-Acc across 2 or 3 PEs reduces the area by 30% and 37%, respectively. Sharing across more PEs only yields marginal area benefits, while the risk of performance degradation increases. Therefore, we suggest and adopt sharing across two PEs in our final design. Figure 16 shows the performance impact of sharing an FP-Acc across multiple PEs based on ResNet-18. The results show that sharing across two PEs incurs a less than

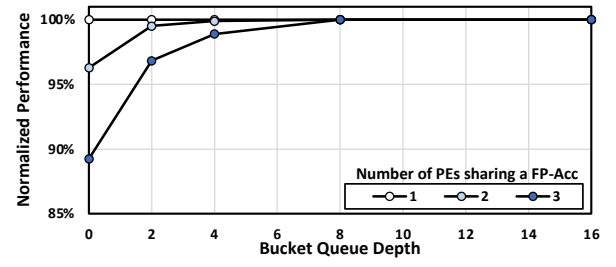


Figure 16: Normalized performance on the first layer of ResNet-18 with different numbers of PEs sharing an FP-Acc. A small bucket queue significantly reduces the performance overheads.

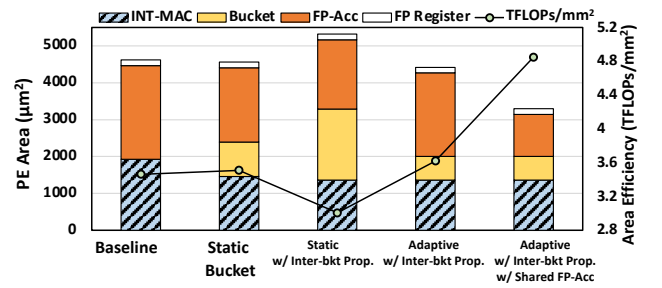


Figure 17: PE area reduction and area efficiency of different schemes

5% performance degradation, which can be further reduced with the help of a bucket queue or an increased number of buckets.

10.6 PE Area Breakdown Comparison

We show the area breakdown comparison of different schemes (with optimized bucket setups determined in Section 10.2) and their corresponding iso-area speedup trends in Figure 17. This analysis clearly shows the source of area reduction. First, the static bucket design (2_{nd} bar) and static bucket design with inter-bucket carry propagation (3_{rd} bar) incur some area overhead due to the insertion of the bucket accumulator. Notably, the areas of MAC and FP-Acc are reduced since the bucket register acts as a pipeline stage separating MAC and FP-Acc circuitry, hence reducing the critical path and circuit area. Second, for the adaptive bucket design with inter-bucket carry propagation (4_{th} bar), the bucket area overhead is significantly reduced since we only have to retain top-K buckets. Finally, the adaptive bucket design with inter-bucket propagation and shared FP-Acc (5_{th} bar) further reduces the area overhead of FP-Acc since we share one FP-Acc among two PEs. In summary, the proposed schemes reduce the PE area by up to 33%, hence improving the iso-area speedup by 1.47 $\times$. We emphasize that this area improvement is achieved in conjunction with significant energy savings.

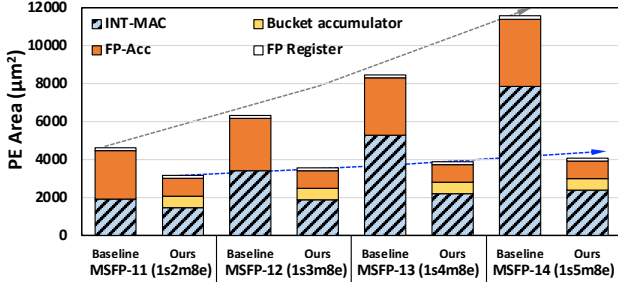


Figure 18: PE area comparison for the baseline BFP-PE and bucket-based BFP-PE with different MSFP configurations

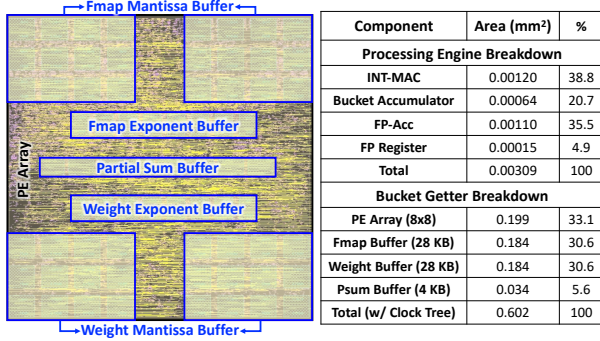


Figure 19: Layout image of *Bucket Getter* and the corresponding area breakdown

10.7 Area of Different Mantissa Bitwidth

Figure 18 analyzes the area of the state-of-the-art BFP-PEs and the bucket-based BFP-PEs with different MSFP configurations, MSFP-11 (1s2m8e), MSFP-12 (1s3m8e), MSFP-13 (1s4m8e), and MSFP-14 (1s5m8e). The first takeaway is that our PE architecture consistently achieves a lower area than that of the baseline PEs. The second takeaway is that the area benefits increase as the mantissa bitwidth increases, with $1.46 \sim 2.84\times$ iso-area speedup. We note that the benefits of our architecture are associated with two main factors: 1) the shared FP-Acc reduces the FP-Acc area, and 2) the MAC area is reduced by inserting a bucket accumulator between the MAC circuitry and the FP-Acc, which reduces the critical path for the MAC circuitry in both units.

10.8 Chip Layout and Area Breakdown

Figure 19 shows an image of the layout of the proposed *Bucket Getter* architecture (Figure 9) using the TSMC 40-nm process. We highlight several important modules: the mantissa buffer, exponent buffer, partial sum buffer, and 8×8 PE array. Then, we summarize the corresponding area breakdown in the right-hand table. Notably, each PE occupies 0.00309 mm^2 of area, where the MAC, bucket accumulator, FP-Acc, and FP registers occupy 0.00120 mm^2 , 0.00064 mm^2 , 0.00110 mm^2 , and 0.00015 mm^2 of area, respectively. The chip occupies an area of 0.602 mm^2 , and the 8×8 PE array occupies 33.1% of the total area. The remaining area is occupied by buffers for the feature map (fmap), weights, and partial sums.

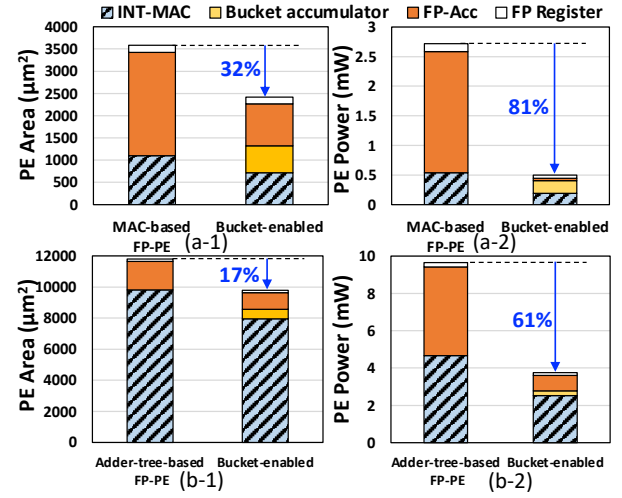


Figure 20: (a-1) Area and (a-2) power comparison of a BF16 MAC-based FP-PE and its bucket-enabled counterpart. (b-1) Area and (b-2) power comparison of a BF16 adder-tree-based FP-PE and its bucket-enabled counterpart

10.9 Area and Power Reduction for FP-PEs

In Figure 11, we show that our proposed architecture can be easily integrated with commercial MAC-based FP-PEs [2, 19] and adder-tree-based FP-PEs [18]. We synthesize these designs and analyze their power with PrimeTimePX using actual patterns. Figure 20(a-1) and Figure 20(a-2) show the area and power of a MAC-based FP-PE and the *Bucket Getter* MAC-based FP-PE. The proposed design reduces the PE area by 32% and reduces the power by 81%. Since the power contribution of FP-Acc is significant for the MAC-based FP-PE, we can reduce this proportion of power and significantly reduce the overall PE power.

Figure 20(b-1) and Figure 20(b-2) show the area and power of an adder-tree-based FP-PE and its bucket-enabled counterpart. Specifically, in the proposed design, the PE area is reduced by 17%, and the power is reduced by 61%. The limited area gain is because of the significant area overhead associated with INT-Mult, alignment, and the large adder tree. Note that our proposed approach is efficient because FP-Acc contributes significantly to the PE power.

11 RELATED WORKS

11.1 Efficient Datatypes for DNNs

The first category of related works customize FP numbers, which optimizes the configuration of floating point for DNNs [9, 11, 46]. For example, NVIDIA has deployed TensorFloat (1s10m8e) on A100 GPUs [33], and Google has proposed Brain Floating Point (1s7m8e) (BF16) [20] for their TPU accelerator [19], as also adopted by the IBM Telum accelerator [2], and Samsung mobile DNN accelerator [18, 36]. However, customized FP architectures still incur a large amount of FP-Acc activity, which limits their energy efficiency.

Another datatype involves BFP format, and a shared exponent is used to scale a vector of mantissas [5, 8, 43–45, 50]. Microsoft has demonstrated that most models require only a 3–5b mantissa to

achieve FP32-comparable accuracy [7, 10]. In the Lake Crest accelerator, Intel has utilized a similar number format, Flexpoint [21]. Recently, some papers have proposed 2-stage scaling [6, 40] and low-bit FP scaling [25] to further improve BFP computation. Although the BFP format can successfully reduce the memory footprint and overhead, power-hungry FP-Acc is needed to accumulate partial sums [7]. In response, our Bucket Getter is the first to target the issue of power-hungry FP-Acc and successfully transform most of the inefficient FP-Acc activities to INT-Acc activities using bucket accumulators.

11.2 Sparsity-Aware and Out-of-Bound-Aware Architectures

There are two main trends for methods that seek to improve BFP or FP DNN accelerators: exploiting sparsity or exploiting out-of-bound terms. Sparsity-aware architecture [12–14, 18, 35, 38, 51] exploits the sparsity associated with ReLU operations, quantization [42], and pruning. Zero input values or zero weights are located to skip the corresponding ineffectual MAC operations. The benefits can be enhanced by further exploiting sub-word sparsity [17] and bit sparsity [3, 29, 41]. In addition, OB-aware architectures avoid MAC computations with exponents that are tiny compared to the existing partial sums [4, 29]. Our work complements these architectures and can boost their energy and area efficiencies.

11.3 Repetition-Aware Architectures

Studies in the last category have explored accelerating DNNs by exploiting repetitive computations. They store some pre-computed results in a group of registers [15, 22, 26, 49]. UCNN [15] exploits repetitive weights and avoids redundant computations with a bit-parallel architecture. UNPU [22] exploits repetitive bit patterns by utilizing LUT-based computations and avoiding redundant computations. FuseKNA [49] also exploits repetitive bit patterns. These methods also employ extra registers inside PEs, but their registers serve as lookup tables instead of partial sum accumulators. In addition, their target data types are not BFP, which involves our identified bottleneck of converting and adding INT-domain MAC results to FP-domain accumulating registers.

12 CONCLUSIONS

This paper presents a novel architecture, **Bucket Getter**, which identifies and mitigates the power and area bottlenecks of FP-Acc. Specifically, we propose 1) a bucket-based accumulator, 2) an inter-bucket carry propagation scheme, 3) an out-of-bound-aware, adaptive and circular bucket accumulator, and 4) a shared FP-Acc. The experimental results based on TSMC 40 nm demonstrate that our proposed **Bucket Getter** architecture reduces energy by up to 57% and yields the iso-area speedup by up to 1.4 $\times$, compared to state-of-the-art BFP engines across seven representative DNN models. Furthermore, **Bucket Getter** reduces the PE area by up to 32% and the PE power by up to 81% compared to the state-of-the-art FP engines.

ACKNOWLEDGMENTS

We thank the reviewers for their constructive comments. We thank NCHC and TSRI for their server services. This work is supported by

the NSTC under grants 111-2823-8-007-001 and 112-2218-E-007-012, the MOE under grant 112QR001N4, the Synopsys Fellowship, and the NSTC Fellowship.

REFERENCES

- [1] Hamzah Abdelaziz, ali shafiee, Jong Hoon Shin, Ardavan Pedram, and Joseph Hassoun. 2021. Rethinking Floating Point Overheads for Mixed Precision DNN Accelerators. In *Proceedings of Machine Learning and Systems*, A. Smola, A. Dimakis, and I. Stoica (Eds.), Vol. 3. 223–239. <https://proceedings.mlsys.org/paper/2021/file/5f93f983524def3dca464469d2cf9f3e-Paper.pdf>
- [2] Ankur Agrawal, Sae Kyu Lee, Joel Silbermann, Matthew Ziegler, Mingu Kang, Swagath Venkataramani, Nianzheng Cao, Bruce Fleischer, Michael Guillorn, Matthew Cohen, Silvia Mueller, Jinwook Oh, Martin Lutz, Jinwook Jung, Siyu Koswatta, Ching Zhou, Vidhi Zalani, James Bonanno, Robert Casatuta, Chia-Yu Chen, Jungwook Choi, Howard Haynie, Alyssa Herbert, Radhika Jain, Monodeep Kar, Kyu-Hyoun Kim, Yulong Li, Zhibin Ren, Scot Rider, Marcel Schaal, Kerstin Schelm, Michael Schuermann, Xiao Sun, Hung Tran, Naigang Wang, Wei Wang, Xin Zhang, Vinay Shah, Brian Curran, Vijayalakshmi Srinivasan, Pong-Fei Lu, Sunil Shukla, Leland Chang, and Kailash Gopalakrishnan. 2021. 9.1 A 7nm 4-Core AI Chip with 25.6TFLOPS Hybrid FP8 Training, 102.4TOPS INT4 Inference and Workload-Aware Throttling. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. 144–146. <https://doi.org/10.1109/ISSCC42613.2021.9365791>
- [3] Jorge Albericio, Alberto Delmás, Patrick Judd, Sayeh Sharify, Gerard O’Leary, Roman Genov, and Andreas Moshovos. 2017. Bit-Pragmatic Deep Neural Network Computing. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 382–394.
- [4] Omar Mohamed Awad, Mostafa Mahmoud, Isak Edo, Ali Hadi Zadeh, Ciaran Bannon, Anand Jayarajan, Gennady Pekhimenko, and Andreas Moshovos. 2021. FPRaker: A Processing Element For Accelerating Neural Network Training. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO ’21)*. Association for Computing Machinery, New York, NY, USA, 857–869. <https://doi.org/10.1145/3466752.3480106>
- [5] Gordon R. Chiu, Andrew C. Ling, Davor Capalija, Andrew Bitar, and Mohamed Saleh Abdelfattah. 2018. Flexibility: FPGAs and CAD in Deep Learning Acceleration. *Proceedings of the 2018 International Symposium on Physical Design* (2018).
- [6] Steve Dai, Rangha Venkatesan, Mark Ren, Brian Zimmer, William Dally, and Bruce Khailany. 2021. VS-Quant: Per-vector Scaled Quantization for Accurate Low-Precision Neural Network Inference. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 3. 873–884. <https://proceedings.mlsys.org/paper/2021/file/f0935e4cd5920aa6c7c996a5ee53a70f-Paper.pdf>
- [7] Bitu Darvish Rouhani, Daniel Lo, Ritchie Zhao, Ming Liu, Jeremy Fowers, Kalin Ovtcharov, Anna Vinogradsky, Sarah Massengill, Lita Yang, Ray Bittner, Alessandro Forin, Haishan Zhu, Taesik Na, Prerak Patel, Shuai Che, Lok Chand Koppaka, XIA SONG, Subhojit Som, Kaustav Das, Saurabh T, Steven Reinhardt, Sitaram Lanka, Eric Chung, and Doug Burger. 2020. Pushing the Limits of Narrow Precision Inference at Cloud Scale with Microsoft Floating Point. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 10271–10281.
- [8] Mario Drumond, Tao LIN, Martin Jaggi, and Babak Falsafi. 2018. Training DNNs with Hybrid Block Floating Point. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc.
- [9] Dionysios Filippas, Christodoulos Peltekis, Giorgos Dimitrakopoulos, and Chrysostomos Nicopoulos. 2023. Reduced-Precision Floating-Point Arithmetic in Systolic Arrays with Skewed Pipelines.
- [10] Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, Logan Adams, Mahdi Ghandi, Stephen Heil, Prerak Patel, Adam Sapek, Gabriel Weisz, Lisa Woods, Sitaram Lanka, Steven Reinhardt, Adrian Caulfield, Eric Chung, and Doug Burger. 2018. A Configurable Cloud-Scale DNN Processor for Real-Time AI. 1–14. <https://doi.org/10.1109/ISCA.2018.00012>
- [11] Sean Fox, Seyedramin Rasoulzadeh, Julian Faraone, david boland, and Philip Leong. 2021. A Block Minifloat Representation for Training Deep Neural Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=6zaTwpNSQ2>
- [12] Ashish Gondimalla, Noah Chesnut, Mithuna Thottethodi, and T. N. Vijaykumar. 2019. SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (Columbus, OH, USA) (MICRO ’52)*. Association for Computing Machinery, New York, NY, USA, 151–165. <https://doi.org/10.1145/3352460.3358291>
- [13] Xin He, Subhankar Pal, Aporva Amarnath, Siying Feng, Dong-Hyeon Park, Austin Rovinski, Haojie Ye, Yuhan Chen, Ronald Dreslinski, and Trevor Mudge. 2020. Sparse-TPU: Adapting Systolic Arrays for Sparse Matrices. In *Proceedings of the 34th ACM International Conference on Supercomputing (Barcelona, Spain) (ICS ’20)*.

- Association for Computing Machinery, New York, NY, USA, Article 19, 12 pages. <https://doi.org/10.1145/3392717.3392751>
- [14] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '52). Association for Computing Machinery, New York, NY, USA, 319–333. <https://doi.org/10.1145/3352460.3358275>
 - [15] Kartik Hegde, Jiyong Yu, Rohit Agrawal, Mengjia Yan, Michael Pellauer, and Christopher W. Fletcher. 2018. UCNN: Exploiting Computational Reuse in Deep Neural Networks via Weight Repetition. In *Proceedings of the 45th Annual International Symposium on Computer Architecture* (Los Angeles, California) (ISCA '18). IEEE Press, 674–687. <https://doi.org/10.1109/ISCA.2018.00062>
 - [16] Wei-Hsing Huang, Tai-Hao Wen, Je-Min Hung, Win-San Khwa, Yun-Chen Lo, Chuan-Jia Jhang, Huna-Hsi Hsu, Yu-Hsiana Chin, Yu-Chiao Chen, Chuna-Chung Lo, Ren-Shuo Liu, Kea-Tiong Tang, Chih-Cheng Hsieh, Yu-Der Chih, Tsung-Yung Chang, and Meng-Fan Chang. 2023. A Nonvolatile AI-Edge Processor with 4MB SLC-MLC Hybrid-Mode ReRAM Compute-in-Memory Macro and 51.4-251TOPS/W. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*.
 - [17] Dongseok Im, Gwangtae Park, Zhiyong Li, Junha Ryu, and Hoi-Jun Yoo. 2023. Sibia: Signed Bit-slice Architecture for Dense DNN Acceleration with Slice-level Sparsity Exploitation. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 69–80. <https://doi.org/10.1109/HPCA56546.2023.10071031>
 - [18] Jun-Woo Jang, Sehwan Lee, Dongyoung Kim, Hyunsun Park, Ali Shafiee Ardestani, Yeongjae Choi, Channoh Kim, Yoojin Kim, Hyeonseok Yu, Hamzah Abdel-Aziz, Jun-Seok Park, Heonsoo Lee, Dongwoo Lee, Myeong Woo Kim, Hanwoong Jung, Heewoo Nam, Dongguen Lim, Seungwon Lee, Joon-Ho Song, Suknam Kwon, Joseph Hassoun, SukHwan Lim, and Changkyu Choi. 2021. Sparsity-Aware and Re-configurable NPU Architecture for Samsung Flagship Mobile SoC. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 15–28. <https://doi.org/10.1109/ISCA52012.2021.00011>
 - [19] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David Patterson. 2021. Ten Lessons From Three Generations Shaped Google's TPUv4: Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 1–14. <https://doi.org/10.1109/ISCA52012.2021.00010>
 - [20] Dhiraj D. Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jijian Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. 2019. A Study of BFLOAT16 for Deep Learning Training. *ArXiv* (2019). [arXiv:1905.12322](https://arxiv.org/abs/1905.12322)
 - [21] Urs Köster, Tristan J. Webb, Xin Wang, Marcel Nassar, Arjun K. Bansal, William H. Constable, Ögüz H. Elilbol, Scott Gray, Stewart Hall, Luke Hornof, Amir Khosrowshahi, Carey Kloss, Ruby J. Pai, and Naveen Rao. 2017. Flexpoint: An Adaptive Numerical Format for Efficient Training of Deep Neural Networks. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 1740–1750.
 - [22] Jinmook Lee, Changhyeon Kim, Sanghoon Kang, Dongjoo Shin, Sangyeob Kim, and Hoi-Jun Yoo. 2018. UNPU: A 50.6TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision. In *2018 IEEE International Solid-State Circuits Conference - (ISSCC)*. 218–220. <https://doi.org/10.1109/ISSCC.2018.8310262>
 - [23] Y. Lo, Y. Tsai, and R. Liu. 2023. LV: Latency-Versatile Floating-Point Engine for High-Performance Deep Neural Networks. *IEEE Computer Architecture Letters* 01 (aug 2023), 1–4. <https://doi.org/10.1109/LCA.2023.3287096>
 - [24] Yun-Chen Lo, Yu-Chun Kuo, Yun-Sheng Chang, Jian-Hao Huang, Jun-Shen Wu, Wen-Chien Ting, Tai-Hsing Wen, and Ren-Shuo Liu. 2019. Physically Tightly Coupled, Logically Loosely Coupled, Near-Memory BNN Accelerator (PTLL-BNN). In *IEEE European Solid State Circuits Conference (ESSCIRC)*.
 - [25] Yun-Chen Lo, Tse-Kuang Lee, and Ren-Shuo Liu. 2023. Block and Subword-Scaling Floating-Point (BSFP): An Efficient Non-Uniform Quantization For Low Precision Inference. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=VWm4o4l3V9e>
 - [26] Yun-Chen Lo and Ren-Shuo Liu. 2023. Bit-Serial Cache: Exploiting Input Bit Vector Repetition to Accelerate Bit-Serial Inference. In *ACM/IEEE Design Automation Conference (DAC)*.
 - [27] Yun-Chen Lo and Ren-Shuo Liu. 2023. Morphable CIM: Improving Operation Intensity and Depthwise Capability for SRAM-CIM Architecture. In *ACM/IEEE Design Automation Conference (DAC)*.
 - [28] Yun-Chen Lo, Chih-Chen Yeh, Jun-Shen Wu, Chia-Chun Wang, Yu-Chih Tsai, Wen-Chien Ting, and Ren-Shuo Liu. 2022. ISSA: Input-Skippable, Set-Associative Computing-in-Memory (SA-CIM) Architecture for Neural Network Accelerators. In *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.
 - [29] Hang Lu, Liang Chang, Chenglong Li, Zixuan Zhu, Shengjian Lu, Yanhuan Liu, and Mingzhe Zhang. 2021. Distilling Bit-Level Sparsity Parallelism for General Purpose Deep Learning Acceleration. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) (MICRO '21). Association for Computing Machinery, New York, NY, USA, 963–976. <https://doi.org/10.1145/3466752.3480123>
 - [30] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2021. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. *Commun. ACM* 65, 1 (dec 2021), 99–106. <https://doi.org/10.1145/3503250>
 - [31] Nebojša Milenković, Vladimir Stanković, and Miljana Milić. 2015. Modular Design Of Fast Leading Zeros Counting Circuit. *Journal of Electrical Engineering* 66 (11 2015), 329–333. <https://doi.org/10.2478/jee-2015-0054>
 - [32] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Iliia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Anshu Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. *CoRR* abs/1906.00091 (2019). [arXiv:1906.00091](https://arxiv.org/abs/1906.00091) <http://arxiv.org/abs/1906.00091>
 - [33] NVIDIA. 2022. TensorFloat-32 in the A100 GPU Accelerates AI Training, HPC up to 20x. <https://blogs.nvidia.com/blog/2020/05/14/tensorfloat-32-precision-format/>
 - [34] OpenAI. 2023. GPT-4 Technical Report. [arXiv:2303.08774](https://arxiv.org/abs/2303.08774) [cs.CL]
 - [35] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Pugliesi, Rangharajan Venkatesan, Bruce Khailany, Joel Emer, Stephen W. Keckler, and William J. Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. 27–40. <https://doi.org/10.1145/3079856.3080254>
 - [36] Jun-Seok Park, Changsoo Park, Suknam Kwon, Hyeon-Seok Kim, Taehoon Jeon, Yesung Kang, Heonsoo Lee, Dongwoo Lee, James Kim, YoungJong Lee, Sangkyu Park, Jun-Woo Jang, SangHyuck Ha, MinSeong Kim, Jihoon Bang, Suk Hwan Lim, and Inyup Kang. 2022. A Multi-Mode 8K-MAC HW-Utilization-Aware Neural Processing Unit with a Unified Multi-Precision Datapath in 4nm Flagship Mobile SoC. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 246–248. <https://doi.org/10.1109/ISSCC42614.2022.9731639>
 - [37] Sai Qian Zhang, Bradley McDanel, and H. T. Kung. 2022. FAST: DNN Training Under Variable Precision Block Floating Point with Stochastic Rounding. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 846–860. <https://doi.org/10.1109/HPCA53966.2022.00067>
 - [38] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 58–70. <https://doi.org/10.1109/HPCA47549.2020.00015>
 - [39] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Mickevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture* (Virtual Event) (ISCA '20). IEEE Press, 446–459. <https://doi.org/10.1109/ISCA45697.2020.00045>
 - [40] Bitar Rouhani, Ritchie Zhao, Venmugil Elango, Rasoul Shafipour, Mathew Hall, Maral Mesmahkoshroshahi, Ankit More, Levi Melnick, Maximilian Golub, Girish Varatkar, Lei Shao, Gaurav Kolhe, Dmitry Melts, Jasmine Klar, Renee L'Heureux, Matt Perry, Doug Burger, Eric Chung, Zhaoxia Deng, Sam Naghshineh, Jongsoo Park, and Maxim Naumov. 2023. With Shared Microexponents, A Little Shifting Goes a Long Way. [arXiv:2302.08007](https://arxiv.org/abs/2302.08007) [cs.LG]
 - [41] Sayeh Sharify, Alberto Delmas Lascorz, Mostafa Mahmoud, Milos Nikolic, Kevin Siu, Dylan Malone Stuart, Zissis Poulos, and Andreas Moshovos. 2019. Laconic Deep Learning Inference Acceleration. In *Proceedings of the 46th International Symposium on Computer Architecture* (Phoenix, Arizona) (ISCA '19). Association for Computing Machinery, New York, NY, USA, 304–317. <https://doi.org/10.1145/3307650.3322255>
 - [42] Gil Shomron, Freddy Gabbay, Samer Kurzum, and Uri Weiser. 2021. Post-Training Sparsity-Aware Quantization. In *Advances in Neural Information Processing Systems*, A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan (Eds.). https://openreview.net/forum?id=qe9z54E_cqE
 - [43] Ilya Solovychik, Ilya Lyubomirsky, Xin Wang, and Sudeep Bhoja. 2022. Block Format Error Bounds and Optimal Block Size Selection. [arXiv:2210.05470](https://arxiv.org/abs/2210.05470) [cs.LG]

- [44] Thierry Tambe, Coleman Hooper, Lillian Pentecost, Tianyu Jia, En-Yu Yang, Marco Donato, Victor Sanh, Paul Whatmough, Alexander M. Rush, David Brooks, and Gu-Yeon Wei. 2021. EdgeBERT: Sentence-Level Energy Optimizations for Latency-Aware Multi-Task NLP Inference. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) (*MICRO '21*). Association for Computing Machinery, New York, NY, USA, 830–844. <https://doi.org/10.1145/3466752.3480095>
- [45] Thierry Tambe, En-Yu Yang, Glenn G. Ko, Yuji Chai, Coleman Hooper, Marco Donato, Paul N. Whatmough, Alexander M. Rush, David Brooks, and Gu-Yeon Wei. 2021. 9.8 A 25mm2 SoC for IoT Devices with 18ms Noise-Robust Speech-to-Text Latency via Bayesian Speech Denoising and Attention-Based Sequence-to-Sequence DNN Speech Recognition in 16nm FinFET. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. 158–160. <https://doi.org/10.1109/ISSCC42613.2021.9366062>
- [46] Thierry Tambe, En-Yu Yang, Zishen Wan, Yuntian Deng, Vijay Janapa Reddi, Alexander Rush, David Brooks, and Gu-Yeon Wei. 2020. Algorithm-Hardware Co-Design of Adaptive Floating-Point Encodings for Resilient Deep Learning Inference. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6. <https://doi.org/10.1109/DAC18072.2020.9218516>
- [47] Swagath Venkataramani, Vijayalakshmi Srinivasan, Wei Wang, Sanchari Sen, Jintao Zhang, Ankur Agrawal, Monodeep Kar, Shubham Jain, Alberto Mannari, Hoang Tran, Yulong Li, Eri Ogawa, Kazuaki Ishizaki, Hiroshi Inoue, Marcel Schaal, Mauricio Serrano, Jungwook Choi, Xiao Sun, Naigang Wang, Chia-Yu Chen, Allison Allain, James Bonano, Nianzheng Cao, Robert Casatuta, Matthew Cohen, Bruce Fleischer, Michael Guillorn, Howard Haynie, Jinwook Jung, Mingu Kang, Kyu-hyoun Kim, Siyu Koswatta, Saekyu Lee, Martin Lutz, Silvia Mueller, Jinwook Oh, Ashish Ranjan, Zhibin Ren, Scot Rider, Kerstin Schelm, Michael Scheuermann, Joel Silberman, Jie Yang, Vidhi Zalani, Xin Zhang, Ching Zhou, Matt Ziegler, Vinay Shah, Moriyoshi Ohara, Pong-Fei Lu, Brian Curran, Sunil Shukla, Leland Chang, and Kailash Gopalakrishnan. 2021. RaPiD: AI Accelerator for Ultra-low Precision Training and Inference. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 153–166. <https://doi.org/10.1109/ISCA52012.2021.00021>
- [48] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. 2022. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. arXiv:2207.02696 [cs.CV]
- [49] Jianxun Yang, Zhao Zhang, Zhuangzhi Liu, Jing Zhou, Leibo Liu, Shaojun Wei, and Shouyi Yin. 2021. FuseKNA: Fused Kernel Convolution based Accelerator for Deep Neural Networks. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 894–907. <https://doi.org/10.1109/HPCA51647.2021.00079>
- [50] Thomas Yeh, Max Sterner, Zerlina Lai, Brandon Chuang, and Alexander Ihler. 2022. Be Like Water: Adaptive Floating Point for Machine Learning. In *International Conference on Machine Learning, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (Eds.). PMLR, 25490–25500. <https://proceedings.mlr.press/v162/yeh22a.html>
- [51] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. 2020. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 261–274. <https://doi.org/10.1109/HPCA47549.2020.00030>